

Sistema de Gestión de Alquiler de Coches

Descripción del Proyecto

Se va a desarrollar un sistema que gestione el alquiler de coches para una agencia. El sistema debe permitir gestionar la información de clientes, coches y alquileres, garantizando que cada coche se alquile únicamente si está disponible.

IMPORTANTE: Se evaluará la claridad del código, la correcta aplicación de los conceptos de POO, la limpieza y la correcta tabulación, así como la documentación interna mediante comentarios en los métodos que requieran explicación.

1. Enums (0.25 puntos)

Define los siguientes enumerados:

- Enum `TipoCombustible` : GASOLINA , DIESEL , ELECTRICO , HIBRIDO
- Enum `TipoCoche` : SEDAN , SUV , FURGONETA , DEPORTIVO

2. Clase Persona (abstracta) (0.75 puntos)

Crea la clase abstracta `Persona` con la siguiente información:

Atributos:

- String `id`
- String `nombre`
- String `apellidos`
- String `dni`
- String `email`
- String `direccion`

- `String telefono`

Constructores:

- Constructor vacío que genere automáticamente el `id` usando `UUID.randomUUID().toString()`
- Constructor con todos los parámetros excepto `id` (que se genera automáticamente con `UUID`)

Métodos:

- Getters y setters para todos los atributos
- `toString()` que muestre todos los atributos
- Sobrescribir `equals()` y `hashCode()` basándose en el atributo `dni`

3. Clase Cliente (hereda de Persona) (0.5 puntos)

Implementa la clase `Cliente` que extienda de `Persona` y añada:

Atributos:

- `String licenciaConducir`
- `LocalDate fechaObtencionLicencia`

Constructores:

- Constructor con todos los parámetros (incluyendo los de la clase padre)
- Constructor copia

Métodos:

- Getters y setters
- `toString()` (debe incluir el número de licencia de conducir y la fecha)

4. Clase Coche (abstracta) (1 punto)

Crea la clase abstracta `Coche` con la siguiente información:

Atributos:

- `Long id`
- `String marca`
- `String modelo`
- `String matricula`
- `int anio`
- `TipoCombustible combustible`
- `TipoCoche tipo`
- `Double precioBase`
- `boolean disponible` (inicialmente `true`)

Constructores:

- Constructor que reciba todos los atributos excepto `id` y `disponible`
 - El `id` se genera automáticamente (puede usarse un contador estático `Long`)
 - El atributo `disponible` se inicializa por defecto a `true`

Métodos:

- Getters y setters para todos los atributos
- `toString()` que muestre la información relevante del coche
- Sobrescribir `equals()` y `hashCode()` basándose en el atributo `id`
- **Método abstracto:** `double calcularPrecioAlquiler(int dias)` - devuelve el precio de alquiler del coche para un número determinado de días

5. Clase CocheEstandar (0.5 puntos)

Crea la clase `CocheEstandar` que extienda de `Coche`.

Atributos adicionales:

- `boolean aireAcondicionado`

Constructores:

- Constructor que reciba todos los atributos necesarios (marca, modelo, matrícula, año, combustible, precioBase, aireAcondicionado)
 - El `TipoCoche` se debe establecer automáticamente a `SEDAN` o `SUV` según el tipo de coche estándar que se esté creando
 - Se debe llamar al constructor de la clase padre `Coche` pasándole los parámetros correspondientes

Métodos:

- Getters, setters y `toString()`
- Implementa `calcularPrecioAlquiler(int dias)` : devuelve el `precioBase` multiplicado por el número de días. Si el coche tiene aire acondicionado, se debe sumar un extra de 5€ por día

6. Clase CochePremium (0.5 puntos)

Crea la clase `CochePremium` que extienda de `Coche` .

Atributos adicionales:

- boolean `gps`
- boolean `seguroExtra`

Constructores:

- Constructor que reciba todos los atributos necesarios (marca, modelo, matrícula, año, combustible, precioBase, `gps`, `seguroExtra`)
 - El `TipoCoche` se debe establecer automáticamente a `DEPORTIVO` o `FURGONETA` según el tipo de coche premium que se esté creando
 - Se debe llamar al constructor de la clase padre `Coche` pasándole los parámetros correspondientes

Métodos:

- Getters, setters y `toString()`
- Implementa `calcularPrecioAlquiler(int dias)` : el precio se calcula como el `precioBase` por día, incrementado en un 20% por la categoría premium. Además, si incluye GPS se suma 10€ diarios y si tiene seguro extra se añade 15€ diarios

7. Interfaz Enviabile y Clase Alquiler (2 puntos)

Primero, crea la interfaz `Enviabile` :

- Define un único método abstracto: `String crearEmail()`

Luego, desarrolla la clase `Alquiler` que gestione el proceso de alquiler de un coche e implemente la interfaz `Enviabile` .

Atributos:

- `Long id` (se genera de forma automática mediante un campo estático que se incremente con cada nueva instancia)
- `Cliente cliente`
- `Coche coche`
- `LocalDate fechaInicio`
- `LocalDate fechaFin`
- `Double precioTotal`

Constructores:

- Constructor que reciba el cliente, el coche, la fecha de inicio y la fecha de fin
 - El `id` se genera automáticamente incrementando un contador estático
 - Se debe calcular el número de días de alquiler usando `ChronoUnit.DAYS.between(fechaInicio, fechaFin)` o un método similar
 - Asignar a `precioTotal` el valor obtenido al llamar a `coche.calcularPrecioAlquiler(dias)`
 - **Validación importante:** Verificar que el coche esté disponible antes de crear el alquiler

Métodos:

- Getters, setters y `toString()`
- **Método adicional:** `void cancelarAlquiler()` - libera el coche (marcando su atributo `disponible` como `true`) y, opcionalmente,

actualiza algún estado interno si fuese necesario

- **String crearEmail()** : genera un String con el resumen del Alquiler: nombre y apellidos del cliente, marca, modelo y matrícula del coche, fechas del alquiler y precio de este

8. Clase Agencia (3 puntos)

Implementa la clase `Agencia` que se encargará de gestionar coches y alquileres.

Atributos:

- `ArrayList<Coche> coches`
- `ArrayList<Alquiler> alquileres`

Constructores:

- Constructor sin parámetros que inicialice ambas colecciones

Métodos:

- `void addCoche(Coche c)` - añade un coche al inventario
- `void removeCoche(Coche c)` - elimina un coche del inventario
- `void listarCochesDisponibles()` - lista sólo coches disponibles
- `void listarAlquileresActivos()` - aquellos cuya fecha de finalización sea mayor a la de hoy
- `Coche buscarCocheDisponibile(TipoCoche tipo)` - devuelve el primer coche disponible del tipo indicado. Si no hay coches disponibles debe lanzar una excepción personalizada `NoHayCochesDisponiblesException` (debes crear esta clase de excepción)
- **boolean realizarAlquiler(Cliente cliente, TipoCoche tipo, LocalDate fechaInicio, LocalDate fechaFin) :**
 - Comprueba que existe un coche disponible del tipo solicitado (usando `buscarCocheDisponibile`)
 - Si lo hay, crea un objeto `Alquiler` para ese coche, actualiza el estado del coche (`disponible = false`) y añade el alquiler a la colección de alquileres

- Devuelve `true` si se realizó el alquiler y `false` en caso contrario
- Debe capturar la excepción `NoHayCochesDisponiblesException` y devolver `false` en ese caso

9. Clase Principal (1.5 puntos)

Crea una clase `Principal` con el método `main` en el que se realice lo siguiente:

- Crea un objeto `Agencia`
- Añade al menos 20 objetos de tipo `Coche` (una mezcla de `CocheEstandar` y `CochePremium`) al inventario con datos variados
- Implementa un menú interactivo usando `Scanner` con las siguientes opciones:
 1. **Listar Coches disponibles** - mostrar todos los coches que están disponibles para alquilar
 2. **Listar Alquileres activos** - mostrar los alquileres cuya fecha de finalización sea posterior a hoy
 3. **Realizar alquiler** - solicitar al usuario:
 - Datos del cliente (nombre, apellidos, DNI, email, dirección, teléfono, licencia de conducir, fecha de obtención)
 - Tipo de coche deseado (SEDAN, SUV, FURGONETA, DEPORTIVO)
 - Fecha de inicio del alquiler
 - Fecha de fin del alquiler
 - Luego buscar un coche disponible y realizar el alquiler, mostrando mensaje de éxito o error
 4. **Mostrar ingresos totales** - calcular y mostrar la suma del `precioTotal` de todos los alquileres cuya `fechaFin` ya haya pasado (es decir, alquileres completados)
 5. **Salir** - finalizar el programa

Nota: El menú debe repetirse hasta que el usuario elija la opción de salir. Incluye validaciones básicas de entrada de datos.

10. Pregunta final (0.5 puntos extra)

En un fichero de texto que deberá entregarse junto al proyecto, explica:

a) Cómo optimizarías la búsqueda de coches disponibles en la Agencia si el número de coches llegara a ser muy elevado (por ejemplo, 10,000 o más).
Un bucle (`for`) directamente podría tardar demasiado tiempo.

b) Reflexiona sobre el impacto en memoria y rendimiento del programa en dicho escenario, y propone posibles soluciones (uso de estructuras de datos, reserva de memoria, indexación, etc.).

EVALUACIÓN

Criterios de evaluación:

- Se valorará especialmente la correcta aplicación de la herencia, la reutilización de código, la encapsulación, y la implementación de métodos que reflejen un adecuado manejo de excepciones y validaciones (por ejemplo, al calcular el número de días de alquiler o al comprobar la disponibilidad de un coche)
- Se recomienda incluir comentarios explicativos en el código para clarificar la funcionalidad de cada método y clase
- Al entregar el proyecto, incluye el código fuente (comprimido o repositorio GitHub), el fichero compilado (por ejemplo, un JAR) y la documentación generada (Javadoc)

Distribución de puntos:

Apartado	Puntos
1. Enums	0.25
2. Clase Persona (abstracta)	0.75
3. Clase Cliente	0.5
4. Clase Coche (abstracta)	1.0

Apartado	Puntos
5. Clase CocheEstandar	0.5
6. Clase CochePremium	0.5
7. Interfaz Enviabile y Clase Alquiler	2.0
8. Clase Agencia	3.0
9. Clase Principal	1.5
TOTAL	10.0
10. Pregunta final (extra)	+0.5

Entregables

1. **Código fuente:** Proyecto Java completo (archivos `.java`), preferiblemente en repositorio individual en Github
2. **Fichero compilado:** JAR ejecutable del proyecto
3. **Documentación Javadoc:** Generada para todas las clases
4. **Fichero de texto:** Con las respuestas a la pregunta final (opcional, para puntos extra)
5. **README (opcional):** Con instrucciones de compilación y ejecución

Consejos y Buenas Prácticas

Organización del código:

- Organiza las clases en paquetes apropiados (ej: `modelos` , `excepciones` , `interfaces` , `app` , `servicios`)
- Utiliza nombres descriptivos para variables y métodos
- Mantén una indentación consistente (4 espacios o 1 tabulador)

Validaciones importantes:

- Validar que las fechas de alquiler sean coherentes (`fechaInicio < fechaFin`)
- Validar que las fechas no sean del pasado al crear un alquiler
- Validar que el DNI tenga formato correcto
- Comprobar que los precios sean positivos
- Verificar que un coche esté disponible antes de alquilarlo

Manejo de excepciones:

- Crear la excepción personalizada `NoHayCochesDisponiblesException` que extienda de `Exception`
- Usar try-catch apropiadamente en el método `realizarAlquiler` de `Agencia`
- Capturar excepciones de entrada de datos en el menú principal

Documentación Javadoc:

- Documentar todas las clases con `/** ... */`
- Incluir `@param`, `@return` y `@throws` donde corresponda
- Explicar el propósito de cada método no trivial

